# Contents



## 1. Project Overview

End-to-end pipeline built around tweets from **119 KOLs** in tech / AI / crypto and related domains:

**Data Mining -> Data Cleaning -> NLP -> Signals -> Backtesting**

| Module | Script | Key Outputs |
|---|---|---|
| Data Mining | `data_mining.py` | `users.csv` industry labels, `history_tweets.csv`, `kol_master.csv` |
| Data Cleaning | `data_cleaning.py` | `history_tweets_clean.csv` (2,343 rows), `tweet_events_clean.csv` (1,103 events) |
| NLP | `nlp.py` | `sentiment_score_nlp` |
| Signals | `signal_testing.py` | IC scan, `signal_best_*.csv`, multi-horizon matrix |
| Backtesting | `backtesting.py` | Equity curve, Sharpe, benchmarks, multi-horizon backtests |

**Labels**: Next-day return of each industry's proxy ETF, `ind_ret_1d`; trading uses **lag1** (signal on day T -> return on T+1) to avoid same-day leakage.

**Headline results** (latest batch, lag1, 5 bp cost):

| Metric | All-industry equal-weight portfolio | Per-industry leg sum (reference) |
|---|---|---|
| Primary signal | `sig_kol_breadth_contrarian` | Same |
| Portfolio / leg days | **51** portfolio days | **89** leg-days |
| Total return | **+6.22%** | +13.96% |
| Sharpe ratio | **2.55** | 2.51 |
| Max draw-down | **-4.24%** | -11.08% |

| Metric | All-industry equal-weight portfolio | Per-industry leg sum (reference) |
|---|---|---|
| IC (lag1) | — | **0.255** (bootstrap **p=0.007**) |

**Sub-sample backtests (exploratory, shorter samples)**:

| Scope | Signal | Trading days | Total return | Sharpe | Max drawdown | IC |
|---|---|---|---|---|---|---|
| AI / LLM | `sig_kol_breadth_contrarian` | 39 | **+5.81%** | **3.08** | -4.51% | **0.40** |
| Crypto | `sig_bear_cowd_strict` | 23 | **+6.63%** | 8.23* | -1.09% | **0.46** |

* Crypto has only 23 trading days; Sharpe / annualized metrics are easily inflated. Use as heterogeneity supplement, not the main conclusion.

## 2. Data Mining

**Script**: `data_mining.py`
**Input**: `users.csv` (119 KOLs: nickname, profile URL, view count, latest tweet)
**Output**: Industry classification, Twikit historical tweets, `kol_master.csv`

### 2.1 Workflow

1. **Industry classification**: Assign KOLs to `ai_tech`, `crypto`, `ai_tools`, etc. from profile / tweet keywords; write back to `users.csv`.
2. **Tweet fetching**: `--fetch-twikit` pulls historical tweets per KOL (supports `--resume`, `--refetch-users`).
3. **Master table**: `--master` / `--from-clean` builds `kol_master.csv` and influence ranking `kol_ranked.csv`.

### 2.2 Industry–Instrument Mapping

| Industry code | Label | Proxy ETF |
|---|---|---|
| `ai_tech` | AI / LLM | QQQ |
| `ai_tools` | AI tools / agents | QQQ |
| `crypto` | Crypto | BTC-USD |
| `semiconductor` | Semiconductors | SOXX |
| `consumer_tech` | Consumer tech | QQQ |

### 2.3 Scale (current batch)

| Item | Count |
|---|---|
| Total KOLs | 119 |
| Raw tweets `history_tweets.csv` | ~2,300+ rows |
| Fetch window | Last ~90 days (expandable) |

### 2.4 Full Code Flow

Corresponds to `data_mining.py`: read KOL list -> rule-based classification -> write `users.csv` -> Twikit fetch -> build `kol_master` from cleaned table.

```
import pandas as pd
from pathlib import Path
from data_mining import (
    read_users,
    rank_kols_by_industry,
    write_industry_to_users,
    export_rankings,
    fetch_posts,
    save_history,
    build_master_from_clean,
```

```
    USERS_CSV,
    HISTORY_CSV,
)

# ---------- 1. Read users.csv, rank by industry ----------
users = read_users()  # parse username from profile URL
ranked, top5 = rank_kols_by_industry(users, top_k=5)
# ranked: industry, views_seed, rank_in_industry, influence_score
write_industry_to_users(ranked)   # write industry / rank columns
export_rankings(ranked, top5)     # -> data/outputs/kol_ranked.csv

# ---------- 2. Twikit historical tweets (cookies required) ----------
usernames = ranked["username"].tolist()
new_tweets = fetch_posts(
    usernames,
    days=90,
    max_per_user=120,
    sleep_user=10,
    resume=True,          # skip users already in history
    retry_failed=True,
)
save_history(new_tweets, replace=False)  # append to history_tweets.csv

# ---------- 3. After cleaning, build master from clean table ----------
# run data_cleaning.py + nlp.py first
master = build_master_from_clean()  # merge NLP, returns, industry labels
master.to_csv("data/outputs/kol_master.csv", index=False, encoding="utf-8-sig")
print(f"kol_master: {len(master)} rows")
```

**Core industry classification logic** (`classify_profile` + `rank_kols_by_industry`):

```
def classify_profile(nickname, username, latest: str) -> str:
    blob = f"{nickname} {username} {latest}".lower()
    for kw in _INDUSTRY_PATTERNS:      # ai, llm, crypto, btc, ...
        if kw in blob:
            return INDUSTRY_MAP[kw]    # -> ai_tech / crypto / ...
    return "unclassified"

ranked = ranked.sort_values(["industry", "views_seed"], ascending=[True, False])
ranked["rank_in_industry"] = ranked.groupby("industry").cumcount() + 1
ranked["influence_score"] = ranked.groupby("industry")["views_seed"].transform(
    lambda s: round(100.0 * s / s.max(), 2) if s.max() else 0.0
)
```

## 3. Data Cleaning

**Script**: `data_cleaning.py`
**Input**: `history_tweets.csv`
**Output**: `history_tweets_clean.csv`, `tweet_events_clean.csv`

### 3.1 Workflow

1. **Text normalization**: Strip URLs / extra whitespace; keep `text_raw`.
2. **Quality filtering**: Low engagement, suspected bots (`freq` / `likes_low`, etc.).
3. **Ticker resolution**: cashtag + industry -> `ticker` / `industry_ticker`; blacklist symbols (e.g. `K`, `POD`) fall back to industry ETF.
4. **Time repair**: `repair_tweet_times()` recovers UTC time from `tweet_id` (Snowflake).
5. **Return alignment**: yfinance daily / 5-minute prices; compute `ind_ret_1d`, etc.

### 3.2 Cleaning Results

| Metric | Value |
|---|---|
| Cleaned tweets | **2,343** rows |
| Valid events (deduplicated) | **1,103** |
| Valid `ind_ret_1d` | **543** (~49%; remainder pending / non-trading days) |
| KOLs in history | **104** / 119 (more fetching can thicken sample) |

### 3.3 Full Code Flow

Corresponds to `data_cleaning.py` `run_cleaning()`: load raw tweets -> clean & annotate -> lexicon sentiment (coarse) -> yfinance return alignment -> export event subset.

```
import pandas as pd
import numpy as np
from data_cleaning import (
    load_history,
    repair_tweet_times,
    annotate_cleaning,
    _apply_lexicon_sentiment,
    aligned_returns,
    select_event_rows,
    enrich_returns_only,
    HISTORY_CLEAN_CSV,
    EVENTS_CLEAN_CSV,
)

# ---------- Full clean (equivalent to python3 data_cleaning.py) ----------
def run_cleaning_pipeline(skip_returns: bool = False):
    df = load_history()                    # history_tweets.csv
```

```
    df = repair_tweet_times(df)            # Snowflake tweet_time repair
    df = annotate_cleaning(df)             # text / industry / ticker / bot flags
    df = _apply_lexicon_sentiment(df)      # sentiment_score (lexicon; NLP overwrites)
    if not skip_returns:
        df = aligned_returns(df)           # yfinance -> ind_ret_1d, etc.
    df.to_csv(HISTORY_CLEAN_CSV, index=False, encoding="utf-8-sig")
    events = select_event_rows(df, drop_bots=True)
    events.to_csv(EVENTS_CLEAN_CSV, index=False, encoding="utf-8-sig")
    return df, events

df, events = run_cleaning_pipeline()

# ---------- Returns-only refresh (equivalent to --returns-only) ----------
# enriched = enrich_returns_only()  # internal: repair -> _refresh_tickers -> aligned_returns
```

**Time repair** (`repair_tweet_times`):

```
# priority: existing CSV time -> snowflake(tweet_id) -> ingested_at
sf = (int(tweet_id) >> 22) + 1288834974657  # ms since epoch
tweet_time = pd.to_datetime(sf, unit="ms", utc=True)
```

**Ticker resolution + return alignment**:

```
# inside annotate_cleaning: body cashtag + industry -> ticker / industry_ticker
ticker, ticker_src, ind_ticker = _resolve_tickers(text_raw, industry)
# blacklist K/POD etc. -> fall back to industry ETF to avoid yfinance errors

# aligned_returns: batch download daily bars; event-day close -> next trading day ind_ret_1d
daily_cache, intra_cache = _price_batch_cache(df)
# per row: anchor-day close, forward 1d/5d/20d; crypto uses UTC calendar
```

**Event selection** (`select_event_rows`): drop `is_bot`, missing industry, invalid time, etc.; used for signals and backtests.

## 4. NLP

**Script**: `nlp.py`
**Input**: `history_tweets_clean.csv`
**Output**: Tweet-level `sentiment_score_nlp` (written to clean / events tables)

### 4.1 Method

- Score tweet text on sentiment, roughly **[-1, 1]** (negative = bearish, positive = bullish).
- Signal layer uses `-sentiment_mean` as a contrarian factor (more bearish sentiment -> stronger signal -> bet on next-day rebound).

### 4.2 Role in Factors

Daily aggregation (by `event_date` × `industry`):

- `sentiment_mean`: mean sentiment for the day
- `bear_posts` / `bull_posts`: bearish / bullish post counts
- `sentiment_dispersion`: disagreement (used in `sig_unanimity_contrarian`, etc.)

### 4.3 Full Code Flow

Corresponds to `nlp.py` `run_nlp()`: read cleaned table -> multi-source sentiment fusion -> write `sentiment_score_nlp` -> sync events table.

```
import pandas as pd
from data_cleaning import repair_tweet_times, select_event_rows, HISTORY_CLEAN_CSV, EVENTS_CLEA
from nlp import enrich_dataframe, composite_scores, lexicon_score, snownlp_score

# ---------- 1. Load and repair timestamps ----------
df = repair_tweet_times(pd.read_csv(HISTORY_CLEAN_CSV))
text_col = "text_clean" if "text_clean" in df.columns else "text"
texts = df[text_col].fillna("").astype(str).tolist()

# ---------- 2. Per tweet: lexicon + SnowNLP weighted fusion ----------
# composite_scores per text:
#   lexicon_score  (CN/EN bull/bear word lists, weight 0.45)
#   snownlp_score  (optional, weight 0.25)
#   transformer    (optional --transformers, weight 0.30)
labels, scores = composite_scores(texts, use_transformers=False)

df = df.copy()
df["sentiment_nlp"] = labels
df["sentiment_score_nlp"] = scores   # [-1, 1]; signals use -sentiment_mean

# ---------- 3. Write back clean + events ----------
df.to_csv(HISTORY_CLEAN_CSV, index=False, encoding="utf-8-sig")
```

```
events = select_event_rows(df, drop_bots=True)
events.to_csv(EVENTS_CLEAN_CSV, index=False, encoding="utf-8-sig")
print(f"NLP done: {len(df)} tweets, {len(events)} events")

# one-liner equivalent: enrich_dataframe(df, use_transformers=False)
```

**Lexicon scoring example** (`lexicon_score`):

```
def lexicon_score(text: str) -> tuple[str, float]:
    t = clean_urls_and_cashtags(text).lower()
    p = sum(1 for w in POS_ZH if w in t)   # bullish word count
    n = sum(1 for w in NEG_ZH if w in t)   # bearish word count
    score = (p - n) / max(p + n, 1)        # normalized to [-1, 1]
    return label_from_score(score), score
```

## 5. Signal Construction & Testing

**Script**: `signal_testing.py`
**Input**: `tweet_events_clean.csv` (with NLP and returns)
**Output**: `signal_leaderboard.csv`, `signal_best_all.csv`, `signal_best_per_industry.csv`

### 5.1 Signal Definitions

Pipeline: **tweet-level scores -> aggregate by** `event_date` **×** `industry` **-> daily derived factors**. Table 1 explains intuition; Table 2 gives formulas.

**Table 1: Signal names and economic intuition**

| Signal | Meaning |
|---|---|
| **`sig_kol_breadth_contrarian`** | More KOLs discussing the industry **and** more bearish sentiment -> higher odds of next-day industry ETF rebound (crowded panic reversal) |
| `sig_bear_crowd` | High **bearish post share** and overall bearish tone -> bet on post-panic rebound |
| `sig_contrarian_top3` | Listen only to **top-3 ranked** KOLs in the industry; trade **against** their sentiment |
| `sig_dispersion_contrarian` | High **disagreement** plus bearish tone -> stronger reversal signal |
| `sig_unanimity_contrarian` | **Low disagreement** plus bearish tone -> consensus easier to fade |
| `sig_bear_crowd_strict` | Panic reversal only when **>50% of posts are bearish**; filters sparse bearish noise |
| `sig_rank1_momentum` | **Follow** sentiment of industry influence #1 KOL (momentum; opposite of main reversal logic) |
| `sig_contrarian` (base) | Tweet-level **sentiment reversal × influence**; building block for most daily reversal signals |

**Table 2: Signal names and daily formulas**

| Signal | Formula |
|---|---|
| **`sig_kol_breadth_contrarian`** | `n_kol * (-sentiment_mean) * influence_sum / n_posts` |
| `sig_bear_crowd` | `bear_ratio * (-sentiment_mean) * influence_sum`, where `bear_ratio = bear_posts / n_posts` |
| `sig_contrarian_top3` | Tweet-level: only when `kol_rank_in_industry <= 3`, `(-sentiment) * influence`; daily: **sum** |
| `sig_dispersion_contrarian` | `(-sentiment_mean) * sentiment_dispersion * influence_sum` |
| `sig_unanimity_contrarian` | `(-sentiment_mean) * influence_sum / (sentiment_dispersion + 0.15)` |

| Signal | Formula |
|---|---|
| `sig_bear_crowd_strict` | If `bear_ratio > 0.5`: `bear_ratio * (-sentiment_mean) * influence_sum`, else **0** |
| `sig_rank1_momentum` | Tweet-level: only when `kol_rank_in_industry == 1`, `sentiment * influence`; daily: **sum** |
| `sig_contrarian` (base) | Tweet-level: `(-sentiment) * influence`; daily: **sum** over tweets |

**Notation**: `sentiment_mean` = daily industry mean sentiment; `influence_sum` = sum of influence that day; `sentiment_dispersion` = cross-tweet sentiment std dev; `n_kol` / `n_posts` = participating KOL count / post count.

**Reversal vs momentum**: Names with `contrarian` / `crowd` mostly predict **next-day opposite move**; `momentum` predicts **next-day same direction**.

### 5.2 Signal Construction Code (full flow)

Corresponds to `signal_testing.py`: `load_events` -> `build_signal_variants` (tweet-level) -> `aggregate_daily_signals` (daily) -> `enrich_daily_signals` (daily derived).

```
import numpy as np
import pandas as pd
from data_cleaning import repair_tweet_times

# ---------- 1. Load events and repair timestamps ----------
events = repair_tweet_times(pd.read_csv("data/outputs/tweet_events_clean.csv"))
events["tweet_time"] = pd.to_datetime(events["tweet_time"], utc=True)
events["event_date"] = events["tweet_time"].dt.date

# merge KOL influence (users.csv / kol_ranked.csv)
ranked = pd.read_csv("data/outputs/kol_ranked.csv")[["username", "influence_score", "kol_rank_i
work = events.merge(ranked, on="username", how="left")
work["influence"] = work["influence_score"].fillna(work.get("views_seed", 1)).clip(lower=1)
work["sentiment"] = pd.to_numeric(work["sentiment_score_nlp"], errors="coerce").fillna(0)
rank = pd.to_numeric(work["kol_rank_in_industry"], errors="coerce").fillna(5)

# ---------- 2. Tweet-level signals (one scalar per tweet) ----------
s, inf = work["sentiment"], work["influence"]
work["sig_contrarian"] = (-s) * inf
work["sig_contrarian_top3"] = np.where(rank <= 3, (-s) * inf, 0.0)
work["sig_rank1_momentum"] = np.where(rank <= 1, s * inf, 0.0)

# ---------- 3. Daily aggregation: event_date × industry ----------
gcols = ["event_date", "industry", "industry_label"]
signal_cols = ["sig_contrarian", "sig_contrarian_top3", "sig_rank1_momentum"]

daily = work.groupby(gcols, as_index=False).agg(
```

```
    n_posts=("tweet_id", "count"),
    n_kol=("username", "nunique"),
    influence_sum=("influence", "sum"),
    sentiment_mean=("sentiment", "mean"),
    bear_posts=("sentiment", lambda x: int((x < -0.1).sum())),
    bull_posts=("sentiment", lambda x: int((x > 0.1).sum())),
    ind_ret_1d=("ind_ret_1d", "mean"),
    **{f"{c}_sum": (c, "sum") for c in signal_cols},
)
disp = work.groupby(gcols)["sentiment"].std().rename("sentiment_dispersion")
daily = daily.merge(disp.reset_index(), on=gcols, how="left")

# ---------- 4. Daily derived signals (formula columns on industry panel) ----------
sm = daily["sentiment_mean"].fillna(0)
inf = daily["influence_sum"].fillna(1.0)
n = daily["n_posts"].clip(lower=1)
bear_r = daily["bear_posts"] / n
disp = daily["sentiment_dispersion"].fillna(0)

daily["sig_kol_breadth_contrarian_sum"] = (
    daily["n_kol"].fillna(0) * (-sm) * inf / n
)
daily["sig_bear_crowd_sum"] = bear_r * (-sm) * inf
daily["sig_bear_crowd_strict_sum"] = np.where(bear_r > 0.5, bear_r * (-sm) * inf, 0.0)
daily["sig_dispersion_contrarian_sum"] = (-sm) * disp * inf
daily["sig_unanimity_contrarian_sum"] = (-sm) * inf / (disp + 0.15)
# sig_contrarian_top3_sum / sig_rank1_momentum_sum aggregated in step 3

# ---------- 5. IC test (lag1: day-T signal vs T+1 return) ----------
panel = daily.dropna(subset=["ind_ret_1d"]).sort_values(["industry", "event_date"])
panel["signal_lag1"] = panel.groupby("industry")["sig_kol_breadth_contrarian_sum"].shift(1)
ic = panel[["signal_lag1", "ind_ret_1d"]].corr(method="spearman").iloc[0, 1]
print(f"sig_kol_breadth_contrarian IC(lag1) = {ic:.4f}")
```

### 5.3 Signal Selection (primary vs per-industry exploration)

**Table 1: All-industry universal signals (same formula; main backtest)**

| Role | Signal | IC (lag1) | Valid legs | Use |
|---|---|---|---|---|
| **Primary** | `sig_kol_breadth_contrarian` | **0.255** | 84 | Main backtest, headline conclusion |

| Role | Signal | IC (lag1) | Valid legs | Use |
|---|---|---|---|---|
| Alternative | `sig_bear_crowd` | 0.241 | 84 | Emphasize bearish post share |
| Alternative | `sig_contrarian_top3` | 0.240 | 84 | Top-3 KOL reversal |

These signals use the **same daily formula across all industries** (no industry switching).

**Table 2: Per-industry exploratory signals (IC-optimized per industry; shorter samples)**

| Industry | Signal | IC (lag1) | Valid legs | Meaning |
|---|---|---|---|---|
| AI / LLM (`ai_tech`) | `sig_unanimity_contrarian` | 0.439 | 38 | Strengthen reversal when disagreement is low |
| Crypto (`crypto`) | `sig_bear_crowd_strict` | 0.458 | 22 | Trigger only when bearish share > 50% |
| AI tools (`ai_tools`) | `sig_rank1_momentum` | 0.605 | 22 | #1 KOL momentum only (opposite of main logic) |

Per-industry signals **do not replace the primary signal**; `consumer_tech` and `semiconductor` have too few legs to list.

**Selection principles**:

- **Production / main story**: all-industry `sig_kol_breadth_contrarian` (IC 0.255, bootstrap **p=0.007**).
- **Sub-sample validation (backtested)**: `ai_tech` still works with the primary signal (IC 0.40); `crypto` can try `sig_bear_crowd_strict` (IC 0.46, 23 days).
- **Not headline conclusions**: `ai_tools` (losing in all-industry backtest), `consumer_tech` / `semiconductor` (≤2 legs).

### 5.4 IC Scan (all-industry top, lag1)

| Signal | IC | n_days | Notes |
|---|---|---|---|
| **sig_kol_breadth_contrarian** | **0.255** | 84 | Primary signal |
| sig_bear_crowd | 0.241 | 84 | Panic share |
| sig_contrarian_top3 | 0.240 | 84 | Top-3 |

| Signal | IC | n_days | Notes |
|---|---|---|---|
| sig_dispersion_contrarian | 0.208 | 84 | High-disagreement reversal |

### 5.5 Robustness (diagnostics)

| Metric | Full sample | Train (70%) | Test (30%) |
|---|---|---|---|
| IC (lag1) | 0.255 | 0.096 | 0.314 |
| Bootstrap p-value (two-sided) | **0.007** | — | — |
| IC 95% CI | [0.081, 0.441] | — | — |

## 6. Backtesting

**Script**: `backtesting.py`
**Primary signal**: `sig_kol_breadth_contrarian`
**Mode**: `lag1` | **Cost**: 5 bp one-way | **Return column**: `ind_ret_1d`

### 6.1 Full-Sample Performance (core table)

**Equal-weight portfolio** = each trading day, average PnL across active industry legs, then compound into one equity curve (**use this for external reporting**).

| Metric | Equal-weight (full) | Equal-weight (test) | Per-industry leg sum (full) |
|---|---|---|---|
| Trading / portfolio days | **51** | **16** | 89 leg-days |
| **Total return** | **+6.22%** | **+2.13%** | +13.96% |
| **Annualized return** | **+34.8%** | **+39.3%** | +44.8% |
| **Annualized vol** | **13.7%** | 14.6% | 17.9% |
| **Sharpe ratio** | **2.55** | **2.69** | 2.51 |
| **Max drawdown** | **-4.24%** | -3.01% | -11.08% |
| **Win rate** | **52.9%** | 56.3% | 51.7% |
| IC (lag1) | — | — | **0.255** |
| Avg daily PnL | +0.122% | +0.136% | +0.153% |

**Train / test split (per-industry leg basis, 70% / 30%)**

| Split | Total return | Ann. return | Sharpe | Max drawdown | IC |
|---|---|---|---|---|---|
| Train (46 legs) | +4.94% | +30.3% | 2.01 | -4.32% | 0.102 |
| Test (43 legs) | +8.59% | +62.1% | 3.01 | -7.07% | 0.314 |

**Sensitivity (diagnostics)**: Excluding `crypto`, all-industry equal-weight return is ~**+0.99%** (alpha partly crypto-dependent); full-sample diagnostic portfolio ~**+6.50%** (cost convention differs).

### 6.2 Industry Breakdown Under All-Industry Backtest (unified primary signal)

All-industry command: `python3 backtesting.py --all --signal sig_kol_breadth_contrarian --mode lag1`

| Industry | Legs | Total return | Sharpe | Max drawdown | IC | Win rate | Notes |
|---|---|---|---|---|---|---|---|
| AI / LLM | 39 | +5.81% | 3.08 | -4.51% | 0.40 | 56.4% | Main contributor |
| Crypto | 23 | +11.26% | 11.73* | -2.63% | 0.29 | 60.9% | Same primary signal |
| AI tools / agents | 23 | **-8.27%** | -3.70 | -9.55% | -0.01 | 34.8% | **Recommend exclusion** |
| Consumer tech / semis | 2 | — | — | — | — | — | Too few samples |

* Crypto Sharpe / annualized metrics inflated in all-industry table due to short sample; reference only.

### 6.3 Per-Industry Dedicated Signal Backtests (latest validation)

IC-scan recommended signals within sub-industries (short samples; **exploratory**):

| Scope | Command | Signal | Trading days | Total return | Sharpe | Max drawdown | IC |
|---|---|---|---|---|---|---|---|
| `ai_tech` | `--industry ai_tech` | `sig_kol_breadth_contrarian` | 39 | **+5.81%** | **3.08** | -4.51% | **0.396** |
| `crypto` | `--industry crypto` | `sig_bear_crowd_strict` | 23 | **+6.63%** | 8.23* | -1.09% | **0.458** |

```
python3 backtesting.py --industry ai_tech --signal sig_kol_breadth_contrarian --mode lag1
python3 backtesting.py --industry crypto --signal sig_bear_crowd_strict --mode lag1
```

**Note**: `ai_tech` test set (12 days) Sharpe ~**-0.57** — split-sample volatility is large; `crypto` test set only 7 days, limited statistical power. **Headline numbers still use Section 6.1 all-industry equal-weight portfolio.**

### 6.4 Trading Strategy & Methodology (signal -> trades)

How backtests map **daily signal -> position -> return -> portfolio equity**; drawdowns in Section 6.5 follow these rules.

#### 6.4.1 Instruments and granularity

| Item | Rule |
|---|---|
| **Instrument** | Industry **proxy ETF** per tweet (e.g. `ai_tech`->QQQ, `crypto`->BTC-USD) |
| **Decision granularity** | Aggregate to **industry-day** by `event_date` × `industry`; multiple tweets same day -> one signal |
| **Return label** | `ind_ret_1d` = event-day close -> **next trading day** close industry ETF return (aligned in cleaning) |

**6.4.2 Signal to position (lag1, main backtest convention)** **Step 1 — Daily signal** (primary signal example):

```
signal_raw(T) = sig_kol_breadth_contrarian_sum
              = n_kol(T) * (-sentiment_mean(T)) * influence_sum(T) / n_posts(T)
```

Higher signal -> broader KOL discussion and more bearish tone -> bet on **next-day rebound**.

**Step 2 — Lag signal one day** (no look-ahead):

```
signal_lag1(T) = signal_raw(T-1)    # shift(1) within each industry group
```

**Step 3 — Long/short direction** (directional, not market-neutral):

```
position_lag1(T) = sign(signal_lag1(T))   # +1 long / -1 short / 0 flat
```

| `signal_lag1` | Interpretation (reversal) | Position | Holding period |
|---|---|---|---|
| **> 0** | Yesterday “crowded panic” | **+1 long** industry ETF | Trading day T close -> T+1 close |
| **< 0** | Yesterday “consensus bullish” | **-1 short** industry ETF | Same |
| **= 0** | No clear direction | **0 flat** | No trade |

**Step 4 — Leg PnL and costs**:

```
pnl_gross(T)  = position_lag1(T) * ind_ret_1d(T)
pnl_net(T)    = pnl_gross(T) - |Δposition| * (5bp / 10000)   # 5 bp one-way on turnover
```

Implementation: `signal_testing.build_daily_panel` and `finalize_panel` (per-industry cumulative equity).

**6.4.3 Portfolio construction (all-industry `--all`)** Each trading day `T`:

1. For all industry legs with signal and `ind_ret_1d` on day T, compute `pnl_net`;
2. **Equal-weight average**: `pnl_combo(T) = mean(pnl_net over industries)`;
3. Portfolio equity: `equity(T) = cumprod(1 + pnl_combo)`.

No cross-industry hedging or cap-weighting; each leg trades its industry ETF independently, then averaged daily.

#### 6.4.4 Timeline example (aligned with drawdown)

```
After T-1 close: compute signal_raw(T-1) from T-1 KOL tweets
Before T open:   set position_lag1(T) = sign(signal_raw(T-1))
T close->T+1 close: realize ind_ret_1d(T), record pnl_lag1(T)
```

**2026-05-07 ~ 05-09 drawdown** (portfolio trough -4.24%): multiple industries long or short simultaneously, but `ind_ret_1d` moved against `position` (plus turnover costs); equal-weight portfolio three consecutive down days; **05-15** single-day multi-industry positive PnL recovered prior peak.

#### 6.4.5 vs `contemp` mode (reference only)

| Mode | Signal usage | Purpose |
|---|---|---|
| **lag1 (main)** | **Yesterday's** signal trades **today's** realized return | Tradable; no same-day leakage |
| contemp | Same-day signal × same-day return | Fitting upper bound only; **not for headline conclusions** |

```python
# signal_testing.py - core lines
daily["signal_lag1"] = daily.groupby("industry")["signal_raw"].shift(1)
daily["position_lag1"] = np.sign(daily["signal_lag1"]).fillna(0)
daily["pnl_lag1"] = daily["position_lag1"] * daily["ind_ret_1d"]
```

### 6.5 Maximum Drawdown & Dates

Drawdowns computed under **Section 6.4 lag1 + equal-weight portfolio + 5 bp** (`strategy_equity_combined.csv`

**Portfolio equity**

| Item | Value |
|---|---|
| **Max drawdown** | **-4.24%** |
| Peak date | **2026-04-20** (equity ~1.061) |
| Trough date | **2026-05-09** (equity ~1.016) |
| Recovery to prior peak | **2026-05-15** |

**Worst trading days near trough**

| Date | Daily portfolio PnL | Drawdown from peak |
|---|---|---|
| 2026-05-09 | -0.80% | **-4.24%** (deepest) |
| 2026-05-08 | -0.94% | -3.46% |
| 2026-05-07 | -1.30% | — |

| Date | Daily portfolio PnL | Drawdown from peak |
|---|---|---|
| 2026-05-03 | -0.78% | -2.62% |

**Early drawdown**: 2026-02-17 -> 2026-02-23, ~**-2.42%** (only 1 industry traded; limited reference value).

**Per-industry leg drawdown (peak -> trough)**

| Industry | Max drawdown | Peak | Trough |
|---|---|---|---|
| AI / LLM | -4.51% | 2026-05-04 | 2026-05-10 |
| Crypto | -2.63% | 2026-05-06 | 2026-05-09 |
| AI tools / agents | -9.55% | 2026-02-17 | 2026-05-14 |

### 6.6 Full Code Flow

Corresponds to `backtesting.py` `run_strategy()`: events -> tweet signals -> daily panel -> lag1 positions -> fees -> equal-weight equity -> performance stats.

```
import numpy as np
import pandas as pd
from signal_testing import (
    load_events,
    build_signal_variants,
    build_daily_panel,
    finalize_panel,
    build_combined_portfolio,
    performance_stats,
    performance_stats_combined,
    time_split,
    backtest_by_industry,
)
from backtesting import filter_tweets

SIGNAL = "sig_kol_breadth_contrarian"
RET_COL = "ind_ret_1d"
COST_BPS = 5.0
MODE = "lag1"

# ---------- 1. Load events + tweet-level signals ----------
raw = load_events()
tweets = build_signal_variants(raw)
tweets = filter_tweets(tweets, industry=None, max_rank=None)  # all industries

# ---------- 2. Daily panel: day-T signal; fwd_ret = ind_ret_1d on row (already T+1 return) --
panel = build_daily_panel(tweets, SIGNAL, ret_col=RET_COL)
# inside build_daily_panel:
```

```
#   aggregate_daily_signals -> enrich_daily_signals
#   signal_lag1 = groupby(industry).shift(1)   # yesterday's signal
#   position_lag1 = sign(signal_lag1)
#   pnl_lag1 = position_lag1 * fwd_ret

pnl_col = "pnl_lag1"
pos_col = "position_lag1"
panel = finalize_panel(panel, pnl_col, pos_col, COST_BPS, MODE, SIGNAL)
# finalize_panel: per-industry turnover fees -> pnl_net; per-industry equity_industry

# ---------- 3. Equal-weight portfolio equity ----------
combined = build_combined_portfolio(panel)
# each event_date: average pnl_net across industries -> one portfolio curve

# ---------- 4. Performance metrics (return, Sharpe, drawdown, IC) ----------
train, test = time_split(panel, train_ratio=0.7)
stats_full = performance_stats(panel, pnl_col, pos_col, split="full", net_pnl=True)
stats_test = performance_stats(test, pnl_col, pos_col, split="test", net_pnl=True)
comb_stats = performance_stats_combined(combined, split="combined_full")
by_ind = backtest_by_industry(panel, pnl_col, pos_col)

print("Portfolio total return:", comb_stats["total_return"])
print("Portfolio Sharpe:", comb_stats["sharpe"])
print("Portfolio max drawdown:", comb_stats["max_drawdown"])
print("Full-sample IC:", stats_full.get("ic_signal_fwd_ret"))
```

**lag1 trading logic** (no look-ahead):

```
daily["signal_raw"] = daily["sig_kol_breadth_contrarian_sum"]
daily["signal_lag1"] = daily.groupby("industry")["signal_raw"].shift(1)
daily["position_lag1"] = np.sign(daily["signal_lag1"]).fillna(0)
daily["pnl_lag1"] = daily["position_lag1"] * daily["ind_ret_1d"]   # label is already next-day
net = pnl - turnover * (cost_bps / 10000)   # 5 bp on turnover
equity = (1 + net).cumprod()
```

### 6.7 Output Files

| File | Contents |
|---|---|
| `strategy_backtest_report_all.csv` | Return, Sharpe, drawdown, IC summary |
| `strategy_equity_combined.csv` | Portfolio equity curve |
| `strategy_daily_trades_all.csv` | Daily × industry trade detail |
| `strategy_backtest_by_industry.csv` | Per-industry performance |
| `signal_diagnostics_report.csv` | Bootstrap, ex-crypto |

### 6.8 Multi-Horizon Signal Scan (frequency × industry)

**Command**:

`python3 signal_testing.py --scan-multi-horizon`

**Design**: 5m / 1h use **lag=0 (contemp)**; 1d / 5d / 20d use **lag=1**. For each `(industry, frequency)` sub-sample, compute IC for all `sig_*` and pick the highest.

**Tweet-level label coverage**: 5m 638 | 1h 631 | 1d 543 | 5d 208 | 20d 46.

**Best IC matrix (by industry)**:

| Industry | 5-min | 1-hour | **1-day** | 5-day | 20-day |
|---|---|---|---|---|---|
| AI / LLM | 0.35 (nlp_lex_disagree) | 0.29 | **0.44** (unanimity) | 0.49 | 0.65* |
| Crypto | 0.40 (rt_contrarian) | 0.09 | **0.46** (bear_strict) | 0.64 | 0.81* |
| AI tools | 0.73* | 0.54* | 0.61* (rank1_momentum) | 0.79* | 0.50* |

* Long-horizon labels like 20d have very few industry-days (≤15); IC is exploratory only.

**Outputs**: `signal_matrix_frequency_industry.csv`, `signal_leaderboard_multi_horizon.csv`.

### 6.9 Multi-Horizon Backtest (matrix cell-by-cell validation)

**Command**:

`python3 backtesting.py --backtest-multi-horizon`

On top of the IC matrix, run each `(industry, frequency, best signal)` with **lag / ret_col / mode consistent with the scan**, validating whether IC converts to PnL.

**1-day frequency (main tradable convention, lag1, 5 bp)**:

| Industry | Best signal | IC | Leg return | Sharpe | Max drawdown |
|---|---|---|---|---|---|
| AI / LLM | `sig_unanimity_contrarian` | 0.44 | **+5.81%** | 3.08 | -4.51% |
| Crypto | `sig_bear_crowd_strict` | 0.46 | **+6.63%** | 8.23 | -1.09% |
| AI tools | `sig_rank1_momentum` | 0.61 | **+8.65%** | 11.04 | -1.61% |

**Other frequencies (exploratory, shorter samples)**:

| Industry | Frequency | Signal | Leg return | Sharpe | Notes |
|---|---|---|---|---|---|
| Crypto | 5-day | `sig_rt_contrarian` | +17.0% | 21.5 | 19 leg-days |
| AI tools | 5-day | `sig_rank1_momentum` | +32.6% | 127* | 19 leg-days; Sharpe easily distorted |

| Industry | Frequency | Signal | Leg return | Sharpe | Notes |
|---|---|---|---|---|---|
| AI / LLM | 1-hour | `sig_strong_buy_trade` | -1.54% | -3.14 | High IC but negative PnL |
| Crypto | 5-min | `sig_rt_contrarian` | +0.03% | 0.20 | Little incremental at short freq |

**Conclusions**:

1. **Tradable headline still on 1-day lag1**; `ai_tech` / `crypto` per-industry bests align with Sections 6.1–6.4.
2. **High IC ≠ profitable** (e.g. `ai_tech` 1h, 5d: IC>0.29 but losing backtests).
3. `ai_tools` **with rank1_momentum (follow #1 KOL)** beats all-industry breadth reversal — industry routing matters.
4. 5m / 1h / 20d constrained by **lag=0 timing** or **small samples**; not headline conclusions.

**Output**: `signal_matrix_backtest.csv`.

### 6.10 Benchmark Comparison (unified convention)

**Command**:

```
python3 backtesting.py --compare-benchmarks --all --ret-col ind_ret_1d --mode lag1
```

All-industry, same `ind_ret_1d`, lag1, 5 bp cost; compare benchmarks and candidate strategies (**equal-weight portfolio metrics primary**):

| Strategy | Portfolio total return | Portfolio Sharpe | Max drawdown | Leg IC |
|---|---|---|---|---|
| Buy-and-hold (long on event days) | +7.49% | 2.93 | -4.75% | — |
| Always flat | 0% | — | 0% | — |
| Random long/short | -10.92% | -3.23 | -12.25% | -0.24 |
| Follow-sentiment baseline | -10.20% | -2.98 | -12.36% | -0.19 |
| Reversal contrarian | +6.65% | 2.75 | -4.20% | 0.19 |

| Strategy | Portfolio total return | Portfolio Sharpe | Max drawdown | Leg IC |
|---|---|---|---|---|
| **Primary breadth×reversal** | **+6.22%** | **2.55** | **-4.24%** | **0.255** |
| ML binary sig_ml_up | -3.93% | -1.43 | -5.99% | 0.04 |
| **Per-industry 1d IC routing** | **+11.75%** | **7.33*** | **-1.93%** | **0.263** |

* Per-industry routing Sharpe easily inflated on 51 portfolio days, but **return and drawdown both beat single primary signal**.

**Benchmark conclusions**:

1. Primary signal **significantly beats** random and follow-sentiment baselines; consistent with contrarian family.

2. Primary signal **slightly underperforms** buy-and-hold on KOL event days (+7.5% vs +6.2%), but **higher IC and interpretable logic**; live deployment can add **trade only on strong signals** to control beta.

3. **ML control weakest** — supports small-sample, interpretable rules.

4. **Per-industry routing** (`ai_tech`->unanimity, `crypto`->bear_strict, `ai_tools`->rank1_momentum) is current best combo; prioritize for productization.

**Output**: `strategy_benchmark_comparison.csv`.

## 7. Limitations & Next Steps

1. **Short sample**: ~84 IC legs, 51 portfolio days; multi-horizon 20d has only 46 tweet-level labels; continue thickening data.

2. **Crypto dependence**: Excluding crypto, portfolio return ~+1% (Section 6.10 buy-and-hold still embeds crypto beta).

3. **IC vs PnL divergence**: Short-horizon / 5d scans can show high IC but negative backtests (Section 6.9); do not select signals on IC alone.

4. **Per-industry routing better but OOS pending**: 1d routed portfolio +11.75%; Sharpe may be overstated on short window; deploy after sample >100 days.

5. **ML / complex models**: `sig_ml_up` benchmark weakest; not primary strategy for now.